

Learn JavaScript & TypeScript

A 14-Day / 28-Hour Job-Ready Curriculum

Two hours a day for two weeks. JavaScript foundations first, then OOP, async, and TypeScript on top. Full depth on the 'why', hands-on exercises every day, a Week 1 Node.js project, and a Week 2 TypeScript capstone. Companion files live in `D:/Git/learning-code/typescript-javascript/`

Week	Focus	Days
Week 1	JavaScript language foundations	Days 1-7
Week 2	OOP, async, TypeScript, testing, tooling	Days 8-14

How To Use This Track

Each day lives in its own folder: dayNN-topic/. Every folder has a lesson.md (read first), an exercises.js/.ts with TODO blanks you fill in and run yourself, and a solutions file to compare against afterward. New to coding entirely? Start with day00-primer/lesson.md before Day 1.

Setup: install Node.js (this track was built and verified against v24). No extra packages until Day 6 (npm basics), Day 11 (the TypeScript compiler), and Day 12 (Jest for testing) -- each lesson tells you exactly what to install when you get there.

Week 1 -- JavaScript Foundations

Day 1: Variables, Types, and Equality

let/const vs var; primitives; undefined vs null; == vs === and why professional code always uses ===; type coercion; truthy/falsy (and how it differs from Python's rules).

Day 2: Control Flow: Conditionals, Loops, Truthy/Falsy

if/else if/else; the ternary; for, while, for...of (values), for...in (object keys); break/continue; switch and the fall-through gotcha.

Day 3: Data Structures: Arrays and Objects, Destructuring, Spread/Rest

Array methods including the critical map/filter/reduce trio; objects as JS's key-value collection; destructuring; spread vs rest (same ... syntax, opposite jobs).

Day 4: Functions: Declarations, Arrow Functions, Scope, Closures, this

Three ways to write a function and when to use each; default/rest parameters; block scope (let/const) vs function scope (var); closures; a real, correct mental model for `this`.

Day 5: Strings, Template Literals, JSON, and Dates

String methods; template literals in depth; JSON.stringify/parse; the Date object and its zero-indexed-months gotcha.

Day 6: Errors, Modules, npm and Packages

try/catch/finally; custom error classes; ES modules (import/export) vs CommonJS (require); npm and package.json.

Day 7: Mini Project: Node.js CLI Task Tracker (Week 1 Review)

A from-scratch command-line to-do app using process.argv and the fs module, combining every Week 1 topic -- verified working end-to-end including all edge cases.

Week 2 -- Job-Ready JavaScript & TypeScript

Day 8: OOP Fundamentals: Classes, Constructors, Encapsulation

class/constructor syntax; this inside methods; private fields (#field) and why encapsulation matters; getters/setters.

Day 9: OOP Advanced: Inheritance, Prototypes, Static Members

extends/super; polymorphism; what's really happening underneath class -- JS's prototype chain; static methods/properties.

Day 10: Asynchronous JavaScript: Callbacks, the Event Loop, Promises, async/await

Why JS needs async at all (the single-threaded event loop); callbacks and 'callback hell'; Promises and .then()/.catch(); async/await; Promise.all for concurrency -- verified with real timing tests.

Day 11: Working With Real Data, and an Introduction to TypeScript

The fetch API for real HTTP requests; why TypeScript exists; setting up tsc; basic type annotations, inference, and why to avoid `any`.

Day 12: TypeScript Deep Dive + Testing with Jest

interface and type; union types and narrowing; generics; setting up and writing real tests with Jest (test.each, async tests, .toThrow) -- verified passing via ts-jest.

Day 13: Practical: Project Structure, npm Scripts, Linting, Git Workflow

Conventional project layout; package.json/npm scripts in depth; the most important tsconfig.json settings (strict mode); ESLint/Prettier; the real git branch + commit + PR workflow.

Day 14: Capstone Project: Expense Tracker (in TypeScript)

A complete CLI application built mostly unassisted: typed OOP models, async file I/O, a custom error class, full Jest test suite, strict TypeScript throughout -- reference solution compiles clean and all 7 tests pass.

What You Should Be Able to Do After Day 14

- Read and reason about real JavaScript/TypeScript code, including closures, this-binding, promises, and async/await.
- Design classes with proper encapsulation, and know when composition/inheritance/plain objects each fit.
- Handle errors deliberately with custom error types instead of silent failures or raw crashes.
- Add TypeScript types (including generics and union types) to catch mistakes before code ever runs.
- Write and run a Jest test suite covering happy paths and edge cases for your own code.
- Set up a project with package.json, tsconfig.json, and a sane git workflow -- the same baseline expected on day one of a JS/TS job.

Full lessons, runnable exercises, and reference solutions for every day are in `D:/Git/learning-code/typescript-javascript/`. Start with `day00-primer/lesson.md`.